

results

May 12, 2026

1 Experiment Runner And Analysis

This notebook is the tracked control panel for running experiments and reviewing results.

What it does: - runs `molssim` and `DegradeMaster` commands via `subprocess` - saves raw command logs under `results/generated/command_logs/` - appends machine-readable entries to `results/experiment_log.jsonl` - loads structured artifacts for plotting - builds 2D and 3D views of training curves and downstream model comparisons

Important repo realities: - `molssim` artifacts include epoch histories. - `DegradeMaster` regression currently writes final metrics and predictions, not a structured epoch-history JSON. - If you run `DegradeMaster` training from this notebook, its stdout logs are saved and can be parsed for per-epoch curves. - `molssim` supports mps on Apple Silicon. Verified downstream code currently selects `cuda` or `cpu`, not `mps`.

```
[1]: from __future__ import annotations

import json
import random
import re
import subprocess
import sys
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.graph_objects as go
import torch
from IPython.display import display
from plotly.subplots import make_subplots

PROJECT_ROOT = Path.cwd().resolve()
if not (PROJECT_ROOT / 'molssim').exists():
    if (PROJECT_ROOT.parent / 'molssim').exists():
        PROJECT_ROOT = PROJECT_ROOT.parent.resolve()

if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
```

```

from molsim.data import DatasetManager
from molsim.models import GraphToVoxelNet, VoxelAutoencoder
from molsim.spatial import VoxelConfig, normalize_voxel, parse_mol2_structure,
    ↪ voxelize_positions
from molsim.metrics import compute_voxel_mse, compute_voxel_overlap

RESULTS_DIR = PROJECT_ROOT / 'results'
GENERATED_DIR = RESULTS_DIR / 'generated'
COMMAND_LOG_DIR = GENERATED_DIR / 'command_logs'
NOTEBOOK_EXPORT_DIR = GENERATED_DIR / 'notebook_exports'
EXPERIMENT_LOG = RESULTS_DIR / 'experiment_log.jsonl'

COMMAND_LOG_DIR.mkdir(parents=True, exist_ok=True)
NOTEBOOK_EXPORT_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

PROJECT_ROOT

```

[1]: PosixPath('/Users/isthismanas/Documents/SVEAssistedModelling')

```

[2]: RESEARCH_EPOCHS = {
    'molsim_baseline': 50,
    'molsim_graph_to_voxel': 40,
    'molsim_mol2_autoencoder': 40,
    'degrademaster_classification': 2000,
    'degrademaster_regression': 400,
}

MOLSIM_COMMANDS = {
    'prepare_qm9': ['python3', 'scripts/prepare_datasets.py', '--dataset-ids',
    ↪ 'qm9', '--export-mol2', '--max-mol2', '0'],
    'train_baseline_qm9': ['python3', 'scripts/train_qm9_baseline.py',
    ↪ '--target', 'gap', '--epochs', str(RESEARCH_EPOCHS['molsim_baseline']),
    ↪ '--batch-size', '64', '--max-samples', '12000', '--device', 'mps'],
    'train_graph_to_voxel': ['python3', 'scripts/train_graph_to_voxel.py',
    ↪ '--epochs', str(RESEARCH_EPOCHS['molsim_graph_to_voxel']), '--batch-size',
    ↪ '32', '--max-samples', '12000', '--grid-size', '16', '--hidden-dim', '128',
    ↪ '--latent-dim', '256', '--dropout', '0.1', '--resolution', '0.5', '--sigma',
    ↪ '0.5', '--device', 'mps', '--mol2-dir', 'data/QM9_mol2',
    ↪ '--checkpoint-path', 'artifacts/graph_to_voxel_qm9.pt', '--artifact-path',
    ↪ 'artifacts/graph_to_voxel_qm9.json'],
}

```

```

    'export_graph_embeddings': ['python3', 'scripts/
↳export_graph_spatial_embeddings.py', '--dataset-id', 'qm9',
↳ '--checkpoint-path', 'artifacts/graph_to_voxel_qm9.pt', '--split', 'all',
↳ '--batch-size', '128', '--device', 'mps', '--output-npz', 'artifacts/
↳degrademaster_graph_spatial_embeddings.npz', '--output-csv', 'artifacts/
↳degrademaster_graph_spatial_embeddings.csv', '--output-json', 'artifacts/
↳degrademaster_graph_spatial_embeddings_meta.json'],
    'train_mol2_autoencoder': ['python3', 'scripts/train_mol2_spatial_encoder.
↳py', '--mol2-dir', 'data/QM9_mol2', '--epochs',
↳str(RESEARCH_EPOCHS['molsim_mol2_autoencoder']), '--batch-size', '32',
↳ '--max-samples', '12000', '--device', 'mps', '--grid-size', '24',
↳ '--resolution', '0.45', '--sigma', '0.5', '--embedding-dim', '256',
↳ '--base-channels', '32', '--dropout', '0.1', '--lr', '1e-3',
↳ '--weight-decay', '1e-6', '--checkpoint-path', 'artifacts/
↳voxel_autoencoder_qm9.pt', '--artifact-path', 'artifacts/
↳voxel_autoencoder_qm9.json'],
    'export_mol2_embeddings': ['python3', 'scripts/
↳export_degrademaster_embeddings.py', '--mol2-dir', 'data/QM9_mol2',
↳ '--checkpoint-path', 'artifacts/voxel_autoencoder_qm9.pt', '--batch-size',
↳ '64', '--device', 'mps', '--output-npz', 'artifacts/
↳degrademaster_mol2_embeddings.npz', '--output-json', 'artifacts/
↳degrademaster_mol2_embeddings_meta.json'],
}

```

```

DOWNSTREAM_COMMANDS = {

```

```

    'classification_baseline': ['python3', 'scripts/
↳run_degrademaster_classification.py', '--config', 'downstream/config/config.
↳yaml'],
    'regression_preprocess': ['python3', 'scripts/
↳run_degrademaster_regression_preprocess.py', '--source-data-root', 'data/
↳PROTAC', '--regression-data-root', 'data/PROTAC_regression', '--name-json',
↳ 'data/PROTAC/name.json', '--protac-csv', 'data/_downloads/
↳PROTAC-8K_extracted/PROTAC-8K/protac.csv'],
    'regression_base': ['python3', 'scripts/run_degrademaster_regression.py',
↳ '--variant', 'base'],
    'regression_two_head': ['python3', 'scripts/run_degrademaster_regression.
↳py', '--variant', 'two_head'],
    'regression_cross_attention': ['python3', 'scripts/
↳run_degrademaster_regression.py', '--variant', 'cross_attention'],
    'regression_pdc50_bounded': ['python3', 'scripts/
↳run_degrademaster_regression.py', '--variant', 'pdc50_bounded'],
    'regression_tabular': ['python3', 'scripts/run_degrademaster_regression.
↳py', '--variant', 'tabular'],
}

```

```

list(MOLSIM_COMMANDS), list(DOWNSTREAM_COMMANDS), RESEARCH_EPOCHS

```

```
[2]: ([ 'prepare_qm9',
        'train_baseline_qm9',
        'train_graph_to_voxel',
        'export_graph_embeddings',
        'train_mol2_autoencoder',
        'export_mol2_embeddings'],
      ['classification_baseline',
        'regression_preprocess',
        'regression_base',
        'regression_two_head',
        'regression_cross_attention',
        'regression_pdc50_bounded',
        'regression_tabular'],
      {'molsim_baseline': 50,
        'molsim_graph_to_voxel': 40,
        'molsim_mol2_autoencoder': 40,
        'degrademaster_classification': 2000,
        'degrademaster_regression': 400})

[3]: def patch_downstream_config_epochs(config_paths: list[Path] | None = None, mode:
      ↪ str = 'Train') -> None:
      try:
          import yaml
      except ModuleNotFoundError as exc:
          raise ModuleNotFoundError('PyYAML is only needed for_
      ↪ patch_downstream_config_epochs(). Install with: pip install pyyaml') from exc

      if config_paths is None:
          config_paths = sorted((PROJECT_ROOT / 'downstream' / 'config').glob('*.*
      ↪ yml')) + sorted((PROJECT_ROOT / 'downstream' / 'config').glob('*.*yaml'))
      if not config_paths:
          print('No downstream config files found to patch yet.')
          return

      for path in config_paths:
          payload = yaml.safe_load(path.read_text(encoding='utf-8')) or {}
          epoch_value = RESEARCH_EPOCHS['degrademaster_classification'] if path.
      ↪ name == 'config.yml' else RESEARCH_EPOCHS['degrademaster_regression']
          payload['epoch'] = int(epoch_value)
          payload['mode'] = str(mode)
          path.write_text(yaml.safe_dump(payload, sort_keys=False),
      ↪ encoding='utf-8')
          print(f'Patched {path} -> epoch={epoch_value}, mode={mode}')

# Run this only after downstream/config/*.yaml exists locally:
# patch_downstream_config_epochs()
```

```

[4]: def append_experiment_log(entry: dict, log_path: Path = EXPERIMENT_LOG) -> None:
    log_path.parent.mkdir(parents=True, exist_ok=True)
    with log_path.open('a', encoding='utf-8') as handle:
        handle.write(json.dumps(entry, ensure_ascii=True) + '\n')

def run_command(name: str, command: list[str], workdir: Path = PROJECT_ROOT,
    family: str = 'manual', check: bool = True) -> subprocess.CompletedProcess:
    started = datetime.now(timezone.utc)
    result = subprocess.run(command, cwd=workdir, text=True,
    capture_output=True)
    finished = datetime.now(timezone.utc)

    log_path = COMMAND_LOG_DIR / f"{started.strftime('%Y%m%dT%H%M%S')}_{name}."
    log"
    with log_path.open('w', encoding='utf-8') as handle:
        handle.write('$ ' + ' '.join(command) + '\n\n')
        handle.write(result.stdout)
        if result.stderr:
            handle.write('\n[stderr]\n')
            handle.write(result.stderr)

    append_experiment_log({
        'timestamp_utc': finished.isoformat(),
        'family': family,
        'name': name,
        'command': command,
        'cwd': str(workdir),
        'returncode': int(result.returncode),
        'log_path': str(log_path.relative_to(PROJECT_ROOT)),
    })

    print(result.stdout)
    if result.stderr:
        print(result.stderr)
    if check and result.returncode != 0:
        raise RuntimeError(f'Command failed: {name} -> {result.returncode}')
    return result

def run_command_group(commands: dict[str, list[str]], family: str) -> None:
    for name, command in commands.items():
        print(f'\n=== Running {name} ===')
        run_command(name=name, command=command, family=family)

```

```

[5]: def safe_show(label: str, fn):
    try:

```

```

    result = fn()
    if result is not None:
        display(result)
except Exception as exc:
    print(f'[skip] {label}: {type(exc).__name__}: {exc}')

```

```

[6]: # Example usage:
# run_command('prepare_qm9', MOLSIM_COMMANDS['prepare_qm9'], family='molsim')
# run_command('train_graph_to_voxel', MOLSIM_COMMANDS['train_graph_to_voxel'],
    ↪ family='molsim')
# run_command_group(DOWNSTREAM_COMMANDS, family='degrademaster')

```

```

[7]: def load_diagnostics_payload() -> dict | None:
    path = PROJECT_ROOT / 'results' / 'model_diagnostics.json'
    return _load_json(path)

def diagnostics_frame() -> pd.DataFrame:
    payload = load_diagnostics_payload()
    if not payload:
        return pd.DataFrame()
    rows = []
    for model_name, rec in payload.get('models', {}).items():
        row = {
            'run_name': model_name,
            'diagnostic_assessment': rec.get('assessment'),
            'diagnostic_notes': ' | '.join(rec.get('notes', [])),
        }
        row.update({f'diagnostic::{k}': v for k, v in rec.get('test_metrics',
    ↪ {}).items()})
        row['diagnostic::epochs'] = rec.get('epochs')
        row['diagnostic::max_samples'] = rec.get('max_samples')
        rows.append(row)
    return pd.DataFrame(rows)

def summarize_current_training_state() -> pd.DataFrame:
    rows = []
    molsim_payloads = collect_molsim_artifacts()
    for name, payload in molsim_payloads.items():
        row = {
            'family': 'molsim',
            'run_name': name,
            'epochs': payload.get('epochs'),
            'max_samples': payload.get('max_samples'),
            'best_epoch': payload.get('best_epoch'),
            'checkpoint_path': payload.get('checkpoint_path'),

```

```

        'last_checkpoint_path': payload.get('last_checkpoint_path'),
    }
    if 'best_val_voxel_mse' in payload:
        row['best_val_voxel_mse'] = payload.get('best_val_voxel_mse')
    if 'best_val_voxel_overlap' in payload:
        row['best_val_voxel_overlap'] = payload.
↪get('best_val_voxel_overlap')
    if 'best_val_recon_mse' in payload:
        row['best_val_recon_mse'] = payload.get('best_val_recon_mse')
        row.update({f'metric::{k}': v for k, v in payload.get('test_metrics',
↪{}).items()})
        rows.append(row)

downstream_df = downstream_metrics_frame()
if not downstream_df.empty:
    for _, rec in downstream_df.iterrows():
        row = {'family': 'degrademaster', 'run_name': rec['run_name'],
↪'epochs': np.nan, 'max_samples': np.nan}
        for col, value in rec.items():
            if col in {'run_name', 'run_dir'}:
                continue
            row[f'metric::{col}'] = value
        rows.append(row)

summary = pd.DataFrame(rows)
diag = diagnostics_frame()
if not diag.empty:
    if summary.empty:
        return diag
    summary = summary.merge(diag, on='run_name', how='outer')
return summary

# Run this after the artifact-discovery helper cells below:
# summarize_current_training_state()

```

```

[8]: def available_summary_metrics() -> list[str]:
    summary = summarize_current_training_state()
    return [c for c in summary.columns if c.startswith('metric::')]

def plot_metric_comparison(metric: str) -> go.Figure:
    summary = summarize_current_training_state()
    if summary.empty:
        raise ValueError('No summary artifacts found yet.')
    if metric not in summary.columns:

```

```

        raise ValueError(f'Metric not found: {metric}. Available:␣
↳{available_summary_metrics()}')

sub = summary[['family', 'run_name', metric]].dropna().sort_values(metric)
fig = go.Figure()
for family_name, family_sub in sub.groupby('family'):
    fig.add_trace(go.Bar(
        x=family_sub[metric],
        y=family_sub['run_name'],
        orientation='h',
        text=np.round(family_sub[metric], 6),
        textposition='outside',
        name=family_name,
    ))
fig.update_layout(
    title=f'Comparison for {metric}',
    xaxis_title=metric,
    yaxis_title='run',
    barmode='group',
    height=max(350, 80 * len(sub)),
    margin=dict(l=120, r=40, t=60, b=40),
)
return fig

def plot_results_heatmap() -> go.Figure:
    summary = summarize_current_training_state()
    if summary.empty:
        raise ValueError('No summary artifacts found yet.')

    metric_cols = available_summary_metrics()
    if not metric_cols:
        raise ValueError('No metric columns available yet.')

    matrix = summary[['run_name', *metric_cols]].set_index('run_name')
    numeric = matrix.astype(float)
    normalized = (numeric - numeric.min()) / (numeric.max() - numeric.min()).
↳replace(0, np.nan)

    fig = go.Figure(data=go.Heatmap(
        z=normalized.values,
        x=[m.replace('metric::', '') for m in normalized.columns],
        y=normalized.index.tolist(),
        text=np.round(numeric.values, 6),
        texttemplate='%{text}',
        colorscale='Viridis',
        colorbar=dict(title='normalized'),
    ))

```



```

))
fig.update_layout(title='Normalized result heatmap (compare shape, not_
↳absolute units)', height=max(350, 70 * len(normalized)))
return fig

def plot_available_results_summary() -> go.Figure:
    metrics = available_summary_metrics()
    if not metrics:
        raise ValueError('No summary metrics available yet.')
    return plot_metric_comparison(metrics[0])

# Run this after the artifact-discovery helper cells below:
# plot_available_results_summary()

```

```

[9]: def _load_json(path: Path) -> dict | None:
    if not path.exists():
        return None
    return json.loads(path.read_text(encoding='utf-8'))

def load_degrademaster_results_registry() -> dict:
    path = PROJECT_ROOT / 'results' / 'degrademaster_results.json'
    if not path.exists():
        return {'classification': {}, 'regression': {}}
    return json.loads(path.read_text(encoding='utf-8'))

def collect_molsim_artifacts() -> dict[str, dict]:
    artifacts = {
        'baseline': PROJECT_ROOT / 'artifacts' / 'baseline_qm9_gap.json',
        'graph_to_voxel': PROJECT_ROOT / 'artifacts' / 'graph_to_voxel_qm9.
↳json',
        'mol2_autoencoder': PROJECT_ROOT / 'artifacts' / 'voxel_autoencoder_qm9.
↳json',
    }
    return {name: data for name, path in artifacts.items() if (data :=_
↳_load_json(path)) is not None}

def molsim_history_frame() -> pd.DataFrame:
    rows = []
    for model_name, payload in collect_molsim_artifacts().items():
        for row in payload.get('history', []):
            item = {'model': model_name}
            item.update(row)

```

```

        rows.append(item)
    return pd.DataFrame(rows)

safe_show('molSIM_history_head', lambda: molSIM_history_frame().head())

```

	model	epoch	train_mse	val_rmse	val_mae	val_r2	train_voxel_mse \
0	baseline	1.0	13.526862	1.762656	1.454921	-0.547167	NaN
1	baseline	2.0	2.494359	1.254110	1.051814	0.216799	NaN
2	baseline	3.0	1.424554	1.027792	0.841830	0.473967	NaN
3	baseline	4.0	1.334025	1.074467	0.865945	0.425105	NaN
4	baseline	5.0	1.284322	1.017746	0.846108	0.484201	NaN

	val_voxel_mse	val_voxel_overlap	train_recon_mse	val_recon_mse
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN

```

[10]: def plot_molSIM_histories_2d() -> go.Figure:
        df = molSIM_history_frame()
        if df.empty:
            raise ValueError('No molSIM history artifacts found under artifacts/.')

        metric_cols = [c for c in df.columns if c not in {'model', 'epoch'}]
        fig = make_subplots(rows=len(metric_cols), cols=1, shared_xaxes=True,
                             subplot_titles=metric_cols)
        for row_idx, metric in enumerate(metric_cols, start=1):
            for model_name, sub in df.groupby('model'):
                fig.add_trace(
                    go.Scatter(x=sub['epoch'], y=sub[metric], mode='lines+markers',
                               name=f'{model_name}:{metric}'),
                    row=row_idx,
                    col=1,
                )
        fig.update_layout(height=max(350, 260 * len(metric_cols)), title='molSIM_
        training histories (2D)')
        return fig

safe_show('plot_molSIM_histories_2d', lambda: plot_molSIM_histories_2d())

```

```

[11]: def plot_molSIM_histories_3d(metric: str = 'train_voxel_mse') -> go.Figure:
        df = molSIM_history_frame()
        if df.empty or metric not in df.columns:
            raise ValueError(f'Metric not available: {metric}')

```

```

    model_order = {name: idx for idx, name in enumerate(sorted(df['model'].
↳unique()))}
    fig = go.Figure()
    for model_name, sub in df.groupby('model'):
        fig.add_trace(
            go.Scatter3d(
                x=sub['epoch'],
                y=[model_order[model_name]] * len(sub),
                z=sub[metric],
                mode='lines+markers',
                name=model_name,
            )
        )
    fig.update_layout(
        title=f'molsim history in 3D: {metric}',
        scene=dict(
            xaxis_title='epoch',
            yaxis_title='model index',
            zaxis_title=metric,
        ),
    )
    return fig

safe_show('plot_molsim_histories_3d', lambda:
↳plot_molsim_histories_3d('train_voxel_mse'))

```

```

[12]: def discover_downstream_metric_files() -> list[Path]:
    return sorted(PROJECT_ROOT.glob('downstream/**/*.final_metrics.json'))

def downstream_metrics_frame() -> pd.DataFrame:
    rows = []
    reg = load_degrademaster_results_registry()
    for rec in reg.get('regression', {}).values():
        row = {'run_dir': str(rec.get('metrics_path', '')), 'run_name': rec.
↳get('run_name', rec.get('tag', 'regression_run'))}
        row.update(rec.get('metrics', {}))
        rows.append(row)
    for path in discover_downstream_metric_files():
        payload = _load_json(path)
        if not payload:
            continue
        row = {'run_dir': str(path.parent.relative_to(PROJECT_ROOT)),
↳'run_name': path.parent.name}
        row.update(payload)

```

```

        rows.append(row)
    df = pd.DataFrame(rows)
    if not df.empty:
        ordered_front = ['run_name', 'run_dir']
        other_cols = [c for c in df.columns if c not in ordered_front]
        df = df[ordered_front + other_cols]
        df = df.drop_duplicates(subset=['run_name'], keep='last')
    return df

def show_downstream_metric_status() -> None:
    df = downstream_metrics_frame()
    if df.empty:
        print('No downstream final_metrics.json files found yet.')
        print('Run DegradeMaster regression first, then rerun this cell.')
        return

    metric_cols = [c for c in df.columns if c not in {'run_name', 'run_dir'}]
    print(f'Found {len(df)} downstream metric file(s).')
    print('Available metric columns:')
    for col in metric_cols:
        print(f'- {col}')

show_downstream_metric_status()
safe_show('downstream_metrics_head', lambda: downstream_metrics_frame().head())

```

Found 12 downstream metric file(s).

Available metric columns:

- mae_dc50_nm
- mae_dmax_pct
- rmse_dc50_nm
- rmse_dmax_pct
- r2_dc50_nm
- r2_dmax_pct
- mean_rmse
- loss_scaled
- loss
- accuracy
- auc
- f1
- precision
- recall
- dataset_root
- dataset_type
- feature
- protac_dim


```

def parse_downstream_epoch_logs() -> pd.DataFrame:
    rows = []
    for path in sorted(COMMAND_LOG_DIR.glob('*regression*.log')):
        text = path.read_text(encoding='utf-8')
        for match in DOWNSTREAM_EPOCH_RE.finditer(text):
            rows.append({
                'log_file': path.name,
                'epoch': int(match.group('epoch')),
                'train_loss_scaled': float(match.group('train_loss')),
                'lr': float(match.group('lr')),
                'mean_rmse': float(match.group('mean_rmse')),
            })
    return pd.DataFrame(rows)

safe_show('downstream_epoch_logs_head', lambda: parse_downstream_epoch_logs().
    head())

```

Empty DataFrame
Columns: []
Index: []

```

[14]: def plot_downstream_metrics_2d(metric: str = 'mean_rmse') -> go.Figure:
        df = downstream_metrics_frame()
        if df.empty:
            raise ValueError('No downstream metrics found. Run DegradeMaster_
    regression first.')
        if metric not in df.columns:
            available = [c for c in df.columns if c not in {'run_name', 'run_dir'}]
            raise ValueError(f'Downstream metric not available: {metric}. Available:
    {available}')

        fig = go.Figure(
            data=[go.Bar(x=df['run_name'], y=df[metric], text=np.round(df[metric],
    4), textposition='outside')]
        )
        fig.update_layout(title=f'DegradeMaster comparison: {metric}',
    xaxis_title='run', yaxis_title=metric)
        return fig

# Example after downstream runs exist:
# plot_downstream_metrics_2d('mean_rmse')

```

```

[15]: def plot_downstream_metrics_3d(x_metric: str = 'rmse_dc50_nm', y_metric: str =
    'rmse_dmax_pct', z_metric: str = 'mean_rmse') -> go.Figure:
        df = downstream_metrics_frame()

```

```

needed = {x_metric, y_metric, z_metric}
if df.empty:
    raise ValueError('No downstream metrics found. Run DegradeMaster_
↳ regression first.')
if not needed.issubset(df.columns):
    available = [c for c in df.columns if c not in {'run_name', 'run_dir'}]
    raise ValueError(f'Downstream metrics missing: {sorted(needed - set(df.
↳ columns))}. Available: {available}')

fig = go.Figure(
    data=[
        go.Scatter3d(
            x=df[x_metric],
            y=df[y_metric],
            z=df[z_metric],
            mode='markers+text',
            text=df['run_name'],
            textposition='top center',
            marker=dict(size=6),
        )
    ]
)
fig.update_layout(
    title='DegradeMaster regression variants in 3D metric space',
    scene=dict(xaxis_title=x_metric, yaxis_title=y_metric,
↳ zaxis_title=z_metric),
)
return fig

# Example after downstream runs exist:
# plot_downstream_metrics_3d()

```

```

[16]: def downstream_sve_comparison_frame() -> pd.DataFrame:
    df = downstream_metrics_frame()
    if df.empty:
        return pd.DataFrame()

    df = df.copy()
    df['comparison_key'] = df['run_name'].str.replace('_baseline', '',
↳ regex=False).str.replace('_with_sve', '', regex=False)
    df['sve_flag'] = np.where(df['run_name'].str.contains('with_sve'),
↳ 'with_sve', np.where(df['run_name'].str.contains('baseline'), 'baseline',
↳ 'other'))
    candidate_cols = [c for c in df.columns if c not in {'run_dir', 'run_name',
↳ 'comparison_key', 'sve_flag'}]
    metric_cols = []

```

```

for c in candidate_cols:
    try:
        pd.to_numeric(df[c])
        metric_cols.append(c)
    except Exception:
        continue

base = df[df['sve_flag'] == 'baseline'][['comparison_key', *metric_cols]].
↳rename(columns={c: f'{c}__baseline' for c in metric_cols})
sve = df[df['sve_flag'] == 'with_sve'][['comparison_key', *metric_cols]].
↳rename(columns={c: f'{c}__with_sve' for c in metric_cols})
merged = base.merge(sve, on='comparison_key', how='inner')

for metric in metric_cols:
    bcol = f'{metric}__baseline'
    scol = f'{metric}__with_sve'
    if bcol in merged.columns and scol in merged.columns:
        merged[f'{metric}__delta'] = merged[scol] - merged[bcol]
return merged

safe_show('downstream_sve_comparison_frame', lambda:
↳downstream_sve_comparison_frame())

```

	comparison_key	mae_dc50_nm__baseline	mae_dmax_pct__baseline	\
0	classification	NaN	NaN	
1	regression_base	202.287272	12.474549	
2	regression_cross_attention	176.870095	10.468439	
3	regression_pdc50_bounded	347.328523	14.406449	
4	regression_tabular	204.536250	9.614509	
5	regression_two_head	195.290420	12.752230	

	rmse_dc50_nm__baseline	rmse_dmax_pct__baseline	r2_dc50_nm__baseline	\
0	NaN	NaN	NaN	
1	452.548902	15.937011	0.174772	
2	463.262836	14.896447	0.135235	
3	987.175616	21.443437	-2.926737	
4	534.484408	14.293537	-0.151100	
5	481.148937	15.773338	0.067171	

	r2_dmax_pct__baseline	mean_rmse__baseline	loss_scaled__baseline	\
0	NaN	NaN	NaN	
1	-0.004647	234.242956	1.310935	
2	0.122261	239.079641	1.203231	
3	-0.818815	504.309527	230.295603	
4	0.191874	274.388972	NaN	
5	0.015882	248.461138	1.256001	

	loss__baseline	...	mean_rmse__delta	loss_scaled__delta	loss__delta	\
0	2.43717	...	NaN	NaN	3.719016	
1	NaN	...	9.249658	0.025650	NaN	
2	NaN	...	-16.080598	0.003454	NaN	
3	NaN	...	-167.604511	0.897382	NaN	
4	NaN	...	0.000000	NaN	NaN	
5	NaN	...	-6.092444	-0.005620	NaN	

	accuracy__delta	auc__delta	f1__delta	precision__delta	recall__delta	\
0	-0.172757	-0.236885	-0.370149	0.294574	-0.697674	
1	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN	NaN	NaN	

	feature__delta	protac_dim__delta
0	0	256.0
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN
5	NaN	NaN

[6 rows x 49 columns]

```
[17]: def plot_downstream_sve_deltas(metric: str = 'mean_rmse') -> go.Figure:
    df = downstream_sve_comparison_frame()
    if df.empty:
        raise ValueError('No paired baseline/with-SVE downstream runs found yet.
    ↪')
    delta_col = f'{metric}__delta'
    if delta_col not in df.columns:
        raise ValueError(f'Metric delta not found: {delta_col}')

    fig = go.Figure(data=[go.Bar(x=df['comparison_key'], y=df[delta_col],
    ↪text=np.round(df[delta_col], 6), textposition='outside')])
    fig.update_layout(title=f'DegradeMaster with-SVE minus baseline: {metric}',
    ↪xaxis_title='run family', yaxis_title='delta')
    return fig

# safe_show('plot_downstream_sve_deltas', lambda:
    ↪plot_downstream_sve_deltas('mean_rmse'))
```

```
[18]: def plot_downstream_epoch_history_3d(metric: str = 'mean_rmse') -> go.Figure:
    df = parse_downstream_epoch_logs()
    if df.empty or metric not in df.columns:
        raise ValueError('No parseable downstream epoch logs found yet. Run
↳training from this notebook first.')

    log_order = {name: idx for idx, name in enumerate(sorted(df['log_file'].
↳unique()))}
    fig = go.Figure()
    for log_file, sub in df.groupby('log_file'):
        fig.add_trace(
            go.Scatter3d(
                x=sub['epoch'],
                y=[log_order[log_file]] * len(sub),
                z=sub[metric],
                mode='lines+markers',
                name=log_file,
            )
        )
    fig.update_layout(
        title=f'DegradeMaster parsed epoch history in 3D: {metric}',
        scene=dict(xaxis_title='epoch', yaxis_title='log index',
↳zaxis_title=metric),
    )
    return fig

# plot_downstream_epoch_history_3d('train_loss_scaled')
```

```
[19]: safe_show('summarize_current_training_state', lambda:
↳summarize_current_training_state())
```

	family	run_name	epochs	max_samples	\
0	molSIM	baseline	50.0	5000.0	
1	NaN	baseline_qm9_gap	NaN	NaN	
2	degrademaster	classification_baseline	NaN	NaN	
3	degrademaster	classification_with_sve	NaN	NaN	
4	molSIM	graph_to_voxel	40.0	12000.0	
5	NaN	graph_to_voxel_qm9	NaN	NaN	
6	NaN	graph_to_voxel_qm9_smoke	NaN	NaN	
7	molSIM	mol2_autoencoder	40.0	12000.0	
8	degrademaster	regression_base_baseline	NaN	NaN	
9	degrademaster	regression_base_with_sve	NaN	NaN	
10	degrademaster	regression_cross_attention_baseline	NaN	NaN	
11	degrademaster	regression_cross_attention_with_sve	NaN	NaN	
12	degrademaster	regression_pdc50_bounded_baseline	NaN	NaN	
13	degrademaster	regression_pdc50_bounded_with_sve	NaN	NaN	
14	degrademaster	regression_tabular_baseline	NaN	NaN	

15	degrademaster	regression_tabular_with_sve	NaN	NaN
16	degrademaster	regression_two_head_baseline	NaN	NaN
17	degrademaster	regression_two_head_with_sve	NaN	NaN
18	NaN	voxel_autoencoder_qm9	NaN	NaN

	best_epoch	checkpoint_path \
0	NaN	None
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	40.0	/Users/isthismanas/Documents/SVEAssistedModell...
5	NaN	NaN
6	NaN	NaN
7	3.0	/Users/isthismanas/Documents/SVEAssistedModell...
8	NaN	NaN
9	NaN	NaN
10	NaN	NaN
11	NaN	NaN
12	NaN	NaN
13	NaN	NaN
14	NaN	NaN
15	NaN	NaN
16	NaN	NaN
17	NaN	NaN
18	NaN	NaN

	last_checkpoint_path	metric::val_rmse \
0	None	0.936556
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	/Users/isthismanas/Documents/SVEAssistedModell...	NaN
5	NaN	NaN
6	NaN	NaN
7	/Users/isthismanas/Documents/SVEAssistedModell...	NaN
8	NaN	NaN
9	NaN	NaN
10	NaN	NaN
11	NaN	NaN
12	NaN	NaN
13	NaN	NaN
14	NaN	NaN
15	NaN	NaN
16	NaN	NaN
17	NaN	NaN
18	NaN	NaN

metric::val_mae metric::val_r2 ... diagnostic_assessment \

0	0.727614	0.517686	...	NaN
1	NaN	NaN	...	needs_review
2	NaN	NaN	...	NaN
3	NaN	NaN	...	NaN
4	NaN	NaN	...	NaN
5	NaN	NaN	...	needs_review
6	NaN	NaN	...	poor
7	NaN	NaN	...	NaN
8	NaN	NaN	...	NaN
9	NaN	NaN	...	NaN
10	NaN	NaN	...	NaN
11	NaN	NaN	...	NaN
12	NaN	NaN	...	NaN
13	NaN	NaN	...	NaN
14	NaN	NaN	...	NaN
15	NaN	NaN	...	NaN
16	NaN	NaN	...	NaN
17	NaN	NaN	...	NaN
18	NaN	NaN	...	weak_but_promising

	diagnostic_notes	diagnostic::val_rmse	\
0		NaN	NaN
1	Negative test R2 means the current baseline is...	0.936556	
2		NaN	NaN
3		NaN	NaN
4		NaN	NaN
5			NaN
6	Voxel overlap is near zero, indicating the mod...		NaN
7		NaN	NaN
8		NaN	NaN
9		NaN	NaN
10		NaN	NaN
11		NaN	NaN
12		NaN	NaN
13		NaN	NaN
14		NaN	NaN
15		NaN	NaN
16		NaN	NaN
17		NaN	NaN
18			NaN

	diagnostic::val_mae	diagnostic::val_r2	diagnostic::epochs	\
0	NaN	NaN	NaN	
1	0.727614	0.517686	50.0	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	
5	NaN	NaN	40.0	

6	NaN	NaN	1.0
7	NaN	NaN	NaN
8	NaN	NaN	NaN
9	NaN	NaN	NaN
10	NaN	NaN	NaN
11	NaN	NaN	NaN
12	NaN	NaN	NaN
13	NaN	NaN	NaN
14	NaN	NaN	NaN
15	NaN	NaN	NaN
16	NaN	NaN	NaN
17	NaN	NaN	NaN
18	NaN	NaN	40.0

	diagnostic::max_samples	diagnostic::val_voxel_mse \
0	NaN	NaN
1	5000.0	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN
5	12000.0	0.014614
6	128.0	0.305581
7	NaN	NaN
8	NaN	NaN
9	NaN	NaN
10	NaN	NaN
11	NaN	NaN
12	NaN	NaN
13	NaN	NaN
14	NaN	NaN
15	NaN	NaN
16	NaN	NaN
17	NaN	NaN
18	12000.0	NaN

	diagnostic::val_voxel_overlap	diagnostic::val_recon_mse
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN
5	0.255163	NaN
6	0.042742	NaN
7	NaN	NaN
8	NaN	NaN
9	NaN	NaN
10	NaN	NaN
11	NaN	NaN

12	NaN	NaN
13	NaN	NaN
14	NaN	NaN
15	NaN	NaN
16	NaN	NaN
17	NaN	NaN
18	NaN	0.000231

[19 rows x 44 columns]

```
[20]: def checkpoint_selection_table() -> pd.DataFrame:
    summary = summarize_current_training_state()
    if summary.empty:
        return pd.DataFrame()

    keep = [
        c for c in [
            'family', 'run_name', 'epochs', 'max_samples', 'best_epoch',
            'best_val_voxel_mse', 'best_val_voxel_overlap',
            'best_val_recon_mse',
            'checkpoint_path', 'last_checkpoint_path',
        ] if c in summary.columns
    ]
    return summary[keep].copy()

safe_show('checkpoint_selection_table', lambda: checkpoint_selection_table())
```

	family	run_name	epochs	max_samples	\
0	molstim	baseline	50.0	5000.0	
1	NaN	baseline_qm9_gap	NaN	NaN	
2	degrademaster	classification_baseline	NaN	NaN	
3	degrademaster	classification_with_sve	NaN	NaN	
4	molstim	graph_to_voxel	40.0	12000.0	
5	NaN	graph_to_voxel_qm9	NaN	NaN	
6	NaN	graph_to_voxel_qm9_smoke	NaN	NaN	
7	molstim	mol2_autoencoder	40.0	12000.0	
8	degrademaster	regression_base_baseline	NaN	NaN	
9	degrademaster	regression_base_with_sve	NaN	NaN	
10	degrademaster	regression_cross_attention_baseline	NaN	NaN	
11	degrademaster	regression_cross_attention_with_sve	NaN	NaN	
12	degrademaster	regression_pdc50_bounded_baseline	NaN	NaN	
13	degrademaster	regression_pdc50_bounded_with_sve	NaN	NaN	
14	degrademaster	regression_tabular_baseline	NaN	NaN	
15	degrademaster	regression_tabular_with_sve	NaN	NaN	
16	degrademaster	regression_two_head_baseline	NaN	NaN	
17	degrademaster	regression_two_head_with_sve	NaN	NaN	
18	NaN	voxel_autoencoder_qm9	NaN	NaN	

	best_epoch	best_val_voxel_mse	best_val_voxel_overlap	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	40.0	0.014862	0.252601	
5	NaN	NaN	NaN	
6	NaN	NaN	NaN	
7	3.0	NaN	NaN	
8	NaN	NaN	NaN	
9	NaN	NaN	NaN	
10	NaN	NaN	NaN	
11	NaN	NaN	NaN	
12	NaN	NaN	NaN	
13	NaN	NaN	NaN	
14	NaN	NaN	NaN	
15	NaN	NaN	NaN	
16	NaN	NaN	NaN	
17	NaN	NaN	NaN	
18	NaN	NaN	NaN	

	best_val_recon_mse	checkpoint_path	\
0	NaN	None	
1	NaN	NaN	
2	NaN	NaN	
3	NaN	NaN	
4	NaN	/Users/isthismanas/Documents/SVEAssistedModell...	
5	NaN	NaN	
6	NaN	NaN	
7	0.000156	/Users/isthismanas/Documents/SVEAssistedModell...	
8	NaN	NaN	
9	NaN	NaN	
10	NaN	NaN	
11	NaN	NaN	
12	NaN	NaN	
13	NaN	NaN	
14	NaN	NaN	
15	NaN	NaN	
16	NaN	NaN	
17	NaN	NaN	
18	NaN	NaN	

	last_checkpoint_path
0	None
1	NaN
2	NaN
3	NaN

```

4 /Users/isthismanas/Documents/SVEAssistedModell...
5                                     NaN
6                                     NaN
7 /Users/isthismanas/Documents/SVEAssistedModell...
8                                     NaN
9                                     NaN
10                                    NaN
11                                    NaN
12                                    NaN
13                                    NaN
14                                    NaN
15                                    NaN
16                                    NaN
17                                    NaN
18                                    NaN

```

```
[21]: safe_show('diagnostics_frame', lambda: diagnostics_frame())
```

```

          run_name diagnostic_assessment \
0      baseline_qm9_gap      needs_review
1      graph_to_voxel_qm9      needs_review
2 graph_to_voxel_qm9_smoke      poor
3      voxel_autoencoder_qm9      weak_but_promising

          diagnostic_notes  diagnostic::val_rmse \
0 Negative test R2 means the current baseline is...      0.936556
1                                     NaN
2 Voxel overlap is near zero, indicating the mod...      NaN
3                                     NaN

diagnostic::val_mae  diagnostic::val_r2  diagnostic::epochs \
0      0.727614      0.517686      50
1      NaN      NaN      40
2      NaN      NaN      1
3      NaN      NaN      40

diagnostic::max_samples  diagnostic::val_voxel_mse \
0      5000      NaN
1      12000      0.014614
2      128      0.305581
3      12000      NaN

diagnostic::val_voxel_overlap  diagnostic::val_recon_mse
0      NaN      NaN
1      0.255163      NaN
2      0.042742      NaN
3      NaN      0.000231

```



```
[22]: def plot_best_checkpoint_metrics() -> go.Figure:
        summary = summarize_current_training_state()
        if summary.empty:
            raise ValueError('No summary artifacts found yet.')

        metric_candidates = [c for c in ['best_val_voxel_mse',
        ↪ 'best_val_voxel_overlap', 'best_val_recon_mse'] if c in summary.columns]
        metric_candidates = [c for c in metric_candidates if summary[c].notna().
        ↪ any()]
        if not metric_candidates:
            raise ValueError('No best-checkpoint metrics available yet.')

        fig = make_subplots(rows=len(metric_candidates), cols=1,
        ↪ subplot_titles=metric_candidates)
        for row_idx, metric in enumerate(metric_candidates, start=1):
            sub = summary[['run_name', metric]].dropna().sort_values(metric)
            fig.add_trace(
                go.Bar(x=sub[metric], y=sub['run_name'], orientation='h', text=np.
        ↪ round(sub[metric], 6), textposition='outside', name=metric),
                row=row_idx,
                col=1,
            )
        fig.update_layout(height=max(320, 240 * len(metric_candidates)),
        ↪ title='Best-checkpoint selection metrics')
        return fig

safe_show('plot_best_checkpoint_metrics', lambda:
        ↪ plot_best_checkpoint_metrics())
```

```
[23]: safe_show('plot_available_results_summary', lambda:
        ↪ plot_available_results_summary())
```

```
[24]: safe_show('available_summary_metrics', lambda: pd.DataFrame({'metric':
        ↪ available_summary_metrics()}))
```

```

                metric
0          metric::val_rmse
1          metric::val_mae
2          metric::val_r2
3          metric::val_voxel_mse
4  metric::val_voxel_overlap
5          metric::val_recon_mse
6          metric::mae_dc50_nm
7          metric::mae_dmax_pct
8          metric::rmse_dc50_nm
9          metric::rmse_dmax_pct
```

```

10         metric::r2_dc50_nm
11     metric::r2_dmax_pct
12         metric::mean_rmse
13     metric::loss_scaled
14         metric::loss
15     metric::accuracy
16         metric::auc
17         metric::f1
18     metric::precision
19         metric::recall
20     metric::dataset_root
21     metric::dataset_type
22         metric::feature
23     metric::protac_dim

```

```
[25]: safe_show('plot_results_heatmap', lambda: plot_results_heatmap())
```

```
[skip] plot_results_heatmap: ValueError: could not convert string to float:
'/Users/isthismanas/Documents/SVEAssistedModelling/data/PROTAC'
```

1.1 Graph-To-Voxel Side-By-Side Sample View

This layer inspects a random QM9 graph sample, runs it through the saved graph-to-voxel checkpoint, and compares the predicted voxel field against the original target voxel field.

If the full checkpoint is missing locally, the notebook falls back to the smoke checkpoint automatically.

```
[26]: def load_graph_to_voxel_bundle():
    preferred = [
        (PROJECT_ROOT / 'artifacts' / 'graph_to_voxel_qm9.pt', PROJECT_ROOT /
        ↪ 'artifacts' / 'graph_to_voxel_qm9.json'),
        (PROJECT_ROOT / 'artifacts' / 'graph_to_voxel_qm9_smoke.pt',
        ↪ PROJECT_ROOT / 'artifacts' / 'graph_to_voxel_qm9_smoke.json'),
    ]
    for ckpt_path, artifact_path in preferred:
        if ckpt_path.exists():
            ckpt = torch.load(ckpt_path, map_location='cpu')
            model_cfg = ckpt['model_config']
            voxel_cfg_raw = ckpt['voxel_config']
            model = GraphToVoxelNet(
                in_channels=int(model_cfg['in_channels']),
                hidden_dim=int(model_cfg['hidden_dim']),
                latent_dim=int(model_cfg['latent_dim']),
                grid_size=int(model_cfg['grid_size']),
                dropout=float(model_cfg['dropout']),
            )
            model.load_state_dict(ckpt['model_state_dict'])
            model.eval()
```

```

        voxel_cfg = VoxelConfig(
            grid_size=int(voxel_cfg_raw['grid_size']),
            resolution=float(voxel_cfg_raw['resolution']),
            sigma=float(voxel_cfg_raw['sigma']),
            use_atomic_weights=bool(voxel_cfg_raw['use_atomic_weights']),
        )
        artifact = _load_json(artifact_path) or {}
        return model, voxel_cfg, artifact, ckpt_path
    raise FileNotFoundError('No graph-to-voxel checkpoint found under artifacts/
↪.')

def reconstruct_graph_sample(sample_index: int | None = None, seed: int = 42) ↪
↪-> dict:
    model, voxel_cfg, artifact, ckpt_path = load_graph_to_voxel_bundle()
    manager = DatasetManager(project_root=PROJECT_ROOT)
    dataset = manager.load('qm9')
    max_samples = int(artifact.get('max_samples', min(2000, len(dataset))))
    rng = random.Random(seed)
    idx = rng.randrange(max_samples) if sample_index is None else ↪
↪int(sample_index)
    data = dataset[idx]
    batch_index = torch.zeros(data.x.shape[0], dtype=torch.long)
    with torch.no_grad():
        pred, latent = model(data.x.float(), data.edge_index, batch_index)
        target = voxelize_positions(data.pos, data.z, voxel_cfg)
        pred_np = pred.squeeze(0).squeeze(0).cpu().numpy()
        target_np = target.squeeze(0).numpy()
    return {
        'sample_index': idx,
        'sample_name': str(getattr(data, 'name', f'qm9_{idx}')),
        'coords': data.pos.cpu().numpy(),
        'atomic_nums': data.z.cpu().numpy(),
        'target_voxel': target_np,
        'pred_voxel': pred_np,
        'latent': latent.squeeze(0).cpu().numpy(),
        'metrics': {
            'checkpoint': str(ckpt_path.relative_to(PROJECT_ROOT)),
            'voxel_mse': compute_voxel_mse(target_np, pred_np),
            'voxel_overlap_0.1': compute_voxel_overlap(target_np, pred_np, ↪
↪threshold=0.1),
            'voxel_overlap_0.25': compute_voxel_overlap(target_np, pred_np, ↪
↪threshold=0.25),
        },
    }

```

```
graph_sample_bundle = reconstruct_graph_sample()
safe_show('graph_sample_metrics', lambda: pd.
↳DataFrame([graph_sample_bundle['metrics']]))
```

```

                                checkpoint  voxel_mse  voxel_overlap_0.1  \
0  artifacts/graph_to_voxel_qm9.pt    0.012705          0.272869

                                voxel_overlap_0.25
0                                0.299618
```

```
[27]: def plot_graph_voxel_side_by_side(sample: dict, threshold: float = 0.1) -> go.
↳Figure:
    def _voxel_to_points_local(voxel: np.ndarray, threshold: float = 0.1) ->
↳tuple[np.ndarray, np.ndarray]:
        coords = np.argwhere(voxel >= threshold)
        values = voxel[voxel >= threshold]
        return coords, values

    coords = sample['coords']
    atomic_nums = sample['atomic_nums']
    atom_colors = {1: 'silver', 6: 'gray', 7: 'blue', 8: 'red', 9: 'green', 15:
↳'orange', 16: 'yellow', 17: 'lime', 35: 'brown', 53: 'purple'}
    colors = [atom_colors.get(int(z), 'pink') for z in atomic_nums]
    target_coords, target_vals = _voxel_to_points_local(sample['target_voxel'],
↳threshold=threshold)
    pred_coords, pred_vals = _voxel_to_points_local(sample['pred_voxel'],
↳threshold=threshold)

    fig = make_subplots(
        rows=2,
        cols=2,
        specs=[[{'type': 'scene'}, {'type': 'scene'}], [{'type': 'xy'}, {'type':
↳'xy'}]],
        subplot_titles=('Input atom point cloud', 'Predicted occupied voxel
↳cloud', 'Target central slice', 'Predicted central slice'),
    )
    fig.add_trace(go.Scatter3d(x=coords[:, 0], y=coords[:, 1], z=coords[:, 2],
↳mode='markers', marker=dict(size=5, color=colors), name='atoms'), row=1,
↳col=1)
    fig.add_trace(go.Scatter3d(x=pred_coords[:, 0], y=pred_coords[:, 1],
↳z=pred_coords[:, 2], mode='markers', marker=dict(size=3, color=pred_vals,
↳colorscale='Plasma', opacity=0.7), name='pred_voxel'), row=1, col=2)
    mid_t = sample['target_voxel'].shape[0] // 2
    mid_p = sample['pred_voxel'].shape[0] // 2
    fig.add_trace(go.Heatmap(z=sample['target_voxel'][:, :, mid_t],
↳coloraxis='coloraxis'), row=2, col=1)
```

```

fig.add_trace(go.Heatmap(z=sample['pred_voxel'][:, :, mid_p],
↳coloraxis='coloraxis'), row=2, col=2)
fig.update_layout(title=f"Graph-to-voxel side-by-side:
↳{sample['sample_name']} | threshold={threshold}",
↳coloraxis=dict(colorscale='Viridis'), showlegend=False)
return fig

safe_show('plot_graph_voxel_side_by_side', lambda:
↳plot_graph_voxel_side_by_side(graph_sample_bundle, threshold=0.1))

```

1.2 Sample-Level Spatial Visualization And Reconstruction

These helpers let you inspect one random QM9 mol2 sample as: - a 3D atomic point cloud - a 3D occupied-voxel cloud - original vs reconstructed central voxel slices - sample reconstruction metrics from the saved autoencoder checkpoint

This is the most direct way to judge whether the learned embedding preserves useful geometry.

```

[28]: def reconstruction_intensity_summary(sample: dict) -> pd.DataFrame:
    voxel = sample['voxel']
    recon = sample['recon']
    rows = [
        {'field': 'original', 'min': float(voxel.min()), 'max': float(voxel.
↳max()), 'mean': float(voxel.mean()), 'std': float(voxel.std())},
        {'field': 'reconstruction', 'min': float(recon.min()), 'max':
↳float(recon.max()), 'mean': float(recon.mean()), 'std': float(recon.std())},
    ]
    return pd.DataFrame(rows)

def suggest_reconstruction_threshold(sample: dict, ratio: float = 0.25, floor:
↳float = 0.005) -> float:
    recon_max = float(sample['recon'].max())
    if recon_max <= 0.0:
        return float(floor)
    return float(max(floor, recon_max * ratio))

def summarize_reconstruction_bundle(sample: dict) -> None:
    print('Sample metrics:')
    for key, value in sample['metrics'].items():
        print(f'- {key}: {value}')
    print('\nIntensity summary:')
    display(reconstruction_intensity_summary(sample))
    print(f"Suggested reconstruction threshold:
↳{suggest_reconstruction_threshold(sample):.6f}")

```

```

def load_voxel_autoencoder_from_artifact(artifact_path: Path | None = None,
    ↪checkpoint_path: Path | None = None):
    artifact_path = artifact_path or (PROJECT_ROOT / 'artifacts' /
    ↪'voxel_autoencoder_qm9.json')
    payload = _load_json(artifact_path)
    if payload is None:
        raise FileNotFoundError(f'Autoencoder artifact not found:
    ↪{artifact_path}')

    ckpt = checkpoint_path or (PROJECT_ROOT / 'artifacts' /
    ↪'voxel_autoencoder_qm9.pt')
    if not ckpt.exists():
        raise FileNotFoundError(f'Autoencoder checkpoint not found: {ckpt}')

    state = torch.load(ckpt, map_location='cpu')
    model_cfg = payload.get('model', state.get('model_config', {}))
    ckpt_model_cfg = state.get('model_config', {})
    model = VoxelAutoencoder(
        grid_size=int(model_cfg.get('grid_size', ckpt_model_cfg.
    ↪get('grid_size'))),
        embedding_dim=int(model_cfg.get('embedding_dim', ckpt_model_cfg.
    ↪get('embedding_dim'))),
        base_channels=int(model_cfg.get('base_channels', ckpt_model_cfg.
    ↪get('base_channels'))),
        dropout=float(model_cfg.get('dropout', ckpt_model_cfg.get('dropout', 0.
    ↪1))),
    )
    model.load_state_dict(state['model_state_dict'])
    model.eval()

    voxel_cfg = payload.get('voxel', {})
    ckpt_voxel_cfg = state.get('voxel_config', {})
    input_normalization = str(voxel_cfg.get('input_normalization',
    ↪ckpt_voxel_cfg.get('input_normalization', 'none')))
    voxel_config = VoxelConfig(
        grid_size=int(voxel_cfg.get('grid_size', model_cfg.get('grid_size',
    ↪ckpt_voxel_cfg.get('grid_size'))),
        resolution=float(voxel_cfg.get('resolution', ckpt_voxel_cfg.
    ↪get('resolution'))),
        sigma=float(voxel_cfg.get('sigma', ckpt_voxel_cfg.get('sigma'))),
        use_atomic_weights=bool(voxel_cfg.get('use_atomic_weights',
    ↪ckpt_voxel_cfg.get('use_atomic_weights', True))),
    )
    payload['_input_normalization'] = input_normalization

```

```

    return model, voxel_config, payload

def sample_random_mol2_path(mol2_dir: Path | None = None, seed: int | None = None) -> Path:
    mol2_dir = mol2_dir or (PROJECT_ROOT / 'data' / 'QM9_mol2')
    files = sorted(mol2_dir.glob('*.mol2'))
    if not files:
        raise FileNotFoundError(f'No mol2 files found under {mol2_dir}')
    rng = random.Random(seed) if seed is not None else random
    return rng.choice(files)

def reconstruct_mol2_sample(mol2_path: Path | None = None):
    model, voxel_config, payload = load_voxel_autoencoder_from_artifact()
    mol2_path = mol2_path or sample_random_mol2_path()
    coords, atomic_nums, bonds = parse_mol2_structure(mol2_path)
    voxel = voxelize_positions(coords, atomic_nums, voxel_config)
    if payload.get('_input_normalization') == 'per_sample_max':
        voxel = normalize_voxel(voxel)
    with torch.no_grad():
        recon, embedding = model(voxel.unsqueeze(0).float())
    recon = recon.squeeze(0).cpu()
    voxel_np = voxel.squeeze(0).numpy()
    recon_np = recon.squeeze(0).numpy()
    metrics = {
        'sample_name': mol2_path.stem,
        'embedding_dim': int(embedding.shape[-1]),
        'voxel_mse': compute_voxel_mse(voxel_np, recon_np),
        'voxel_overlap_0.1': compute_voxel_overlap(voxel_np, recon_np,
threshold=0.1),
        'voxel_overlap_0.25': compute_voxel_overlap(voxel_np, recon_np,
threshold=0.25),
        'artifact_epochs': payload.get('epochs'),
        'artifact_max_samples': payload.get('max_samples'),
    }
    return {
        'path': mol2_path,
        'coords': coords.numpy(),
        'atomic_nums': atomic_nums.numpy(),
        'bonds': bonds,
        'voxel': voxel_np,
        'recon': recon_np,
        'embedding': embedding.squeeze(0).cpu().numpy(),
        'metrics': metrics,
        'voxel_config': voxel_config,
    }

```

```
sample_bundle = reconstruct_mol2_sample()
safe_show('summarize_reconstruction_bundle', lambda:
    ↪summarize_reconstruction_bundle(sample_bundle))
```

Sample metrics:

```
- sample_name: gdb_76865
- embedding_dim: 256
- voxel_mse: 0.00017028254473996942
- voxel_overlap_0.1: 0.9455909943725029
- voxel_overlap_0.25: 0.840000000004923
- artifact_epochs: 40
- artifact_max_samples: 12000
```

Intensity summary:

	field	min	max	mean	std
0	original	0.000000e+00	1.000000	0.015159	0.075730
1	reconstruction	1.578429e-12	0.931908	0.013330	0.076104

Suggested reconstruction threshold: 0.232977

```
[29]: ATOM_COLORS = {1: 'silver', 6: 'gray', 7: 'blue', 8: 'red', 9: 'green', 15:
    ↪'orange', 16: 'yellow', 17: 'lime', 35: 'brown', 53: 'purple'}
```

```
def plot_sample_point_cloud(sample: dict) -> go.Figure:
    coords = sample['coords']
    atomic_nums = sample['atomic_nums']
    bonds = sample['bonds']
    colors = [ATOM_COLORS.get(int(z), 'pink') for z in atomic_nums]

    fig = go.Figure()
    fig.add_trace(go.Scatter3d(
        x=coords[:, 0], y=coords[:, 1], z=coords[:, 2],
        mode='markers',
        marker=dict(size=6, color=colors),
        text=[f'Z={int(z)}' for z in atomic_nums],
        name='atoms',
    ))

    bx, by, bz = [], [], []
    for i, j in bonds:
        bx.extend([coords[i, 0], coords[j, 0], None])
        by.extend([coords[i, 1], coords[j, 1], None])
        bz.extend([coords[i, 2], coords[j, 2], None])
    if bx:
```



```

        fig.add_trace(go.Scatter3d(x=bx, y=by, z=bz, mode='lines',
↳line=dict(color='black', width=4), name='bonds'))

        fig.update_layout(title=f"Atomic point cloud:
↳{sample['metrics']['sample_name']}", scene=dict(aspectmode='data'))
        return fig

safe_show('plot_sample_point_cloud', lambda:
↳plot_sample_point_cloud(sample_bundle))

```

```

[30]: def voxel_to_points(voxel: np.ndarray, threshold: float = 0.1) -> tuple[np.
↳ndarray, np.ndarray]:
        coords = np.argwhere(voxel >= threshold)
        values = voxel[voxel >= threshold]
        return coords, values

def plot_voxel_clouds(sample: dict, threshold: float = 0.1) -> go.Figure:
        orig_coords, orig_vals = voxel_to_points(sample['voxel'],
↳threshold=threshold)
        recon_coords, recon_vals = voxel_to_points(sample['recon'],
↳threshold=threshold)

        fig = make_subplots(rows=1, cols=2, specs=[[{'type': 'scene'}, {'type':
↳'scene'}]], subplot_titles=('Original voxel cloud', 'Reconstructed voxel
↳cloud'))
        if len(orig_coords) > 0:
            fig.add_trace(go.Scatter3d(x=orig_coords[:, 0], y=orig_coords[:, 1],
↳z=orig_coords[:, 2], mode='markers', marker=dict(size=3, color=orig_vals,
↳colorscale='Viridis', opacity=0.7), name='original'), row=1, col=1)
        else:
            fig.add_annotation(text='No original voxels above threshold', x=0.18,
↳y=0.5, xref='paper', yref='paper', showarrow=False)

        if len(recon_coords) > 0:
            fig.add_trace(go.Scatter3d(x=recon_coords[:, 0], y=recon_coords[:, 1],
↳z=recon_coords[:, 2], mode='markers', marker=dict(size=3, color=recon_vals,
↳colorscale='Plasma', opacity=0.7), name='recon'), row=1, col=2)
        else:
            fig.add_annotation(text=f"No reconstructed voxels above
↳threshold\nrecon max={float(sample['recon'].max()):.5f}", x=0.82, y=0.5,
↳xref='paper', yref='paper', showarrow=False)

        fig.update_layout(title=f"Occupied voxel clouds at threshold={threshold}")
        return fig

```

```
safe_show('plot_voxel_clouds', lambda: plot_voxel_clouds(sample_bundle,
↳threshold=0.1))
```

```
[31]: def plot_voxel_clouds_auto(sample: dict, ratio: float = 0.25, floor: float = 0.
↳005) -> go.Figure:
    threshold = suggest_reconstruction_threshold(sample, ratio=ratio,
↳floor=floor)
    return plot_voxel_clouds(sample, threshold=threshold)

safe_show('plot_voxel_clouds_auto', lambda:
↳plot_voxel_clouds_auto(sample_bundle))
```

```
[32]: def plot_voxel_slices(sample: dict) -> go.Figure:
    voxel = sample['voxel']
    recon = sample['recon']
    mid = voxel.shape[0] // 2

    fig = make_subplots(rows=2, cols=2, subplot_titles=('Original XY', 'Recon_
↳XY', 'Absolute Error XY', 'Embedding Histogram'))
    fig.add_trace(go.Heatmap(z=voxel[:, :, mid], coloraxis='coloraxis'), row=1,
↳col=1)
    fig.add_trace(go.Heatmap(z=recon[:, :, mid], coloraxis='coloraxis'), row=1,
↳col=2)
    fig.add_trace(go.Heatmap(z=np.abs(voxel[:, :, mid] - recon[:, :, mid]),
↳coloraxis='coloraxis2'), row=2, col=1)
    fig.add_trace(go.Histogram(x=sample['embedding'], nbinsx=40,
↳name='embedding'), row=2, col=2)
    fig.update_layout(
        title=f"Voxel reconstruction slices:
↳{sample['metrics']['sample_name']}",
        coloraxis=dict(colorscale='Viridis'),
        coloraxis2=dict(colorscale='Magma'),
        showlegend=False,
    )
    return fig

safe_show('plot_voxel_slices', lambda: plot_voxel_slices(sample_bundle))
```

```
[33]: def evaluate_random_reconstruction_samples(n_samples: int = 8, seed: int = 42)
↳-> pd.DataFrame:
    rng = random.Random(seed)
    files = sorted((PROJECT_ROOT / 'data' / 'QM9_mol2').glob('*.*mol2'))
```

```

chosen = rng.sample(files, k=min(n_samples, len(files)))
rows = []
for path in chosen:
    bundle = reconstruct_mol2_sample(path)
    rows.append(bundle['metrics'])
return pd.DataFrame(rows).sort_values('voxel_mse')

sample_eval_df = evaluate_random_reconstruction_samples(n_samples=8, seed=42)
safe_show('sample_eval_df', lambda: sample_eval_df)

```

	sample_name	embedding_dim	voxel_mse	voxel_overlap_0.1 \
7	gdb_116641	256	0.000171	0.963115
4	gdb_133437	256	0.000174	0.985258
5	gdb_129573	256	0.000219	0.963719
6	gdb_126745	256	0.000236	0.947005
1	gdb_113288	256	0.000259	0.941704
3	gdb_69423	256	0.000269	0.969892
2	gdb_103038	256	0.000269	0.946581
0	gdb_57202	256	0.000303	0.949799

	voxel_overlap_0.25	artifact_epochs	artifact_max_samples
7	0.807309	40	12000
4	0.752688	40	12000
5	0.791519	40	12000
6	0.787546	40	12000
1	0.778986	40	12000
3	0.742574	40	12000
2	0.733333	40	12000
0	0.714286	40	12000

1.3 Retrain Autoencoder For Better Reconstruction

The currently saved autoencoder artifact is a weak run (1 epoch, 256 samples). Use the re-search preset below to regenerate a stronger checkpoint before treating reconstruction quality as deployment-ready.

```
[34]: MOLSIM_COMMANDS['train_mol2_autoencoder']
```

```
[34]: ['python3',
'scripts/train_mol2_spatial_encoder.py',
'--mol2-dir',
'data/QM9_mol2',
'--epochs',
'40',
'--batch-size',
'32',
'--max-samples',
```

```
'12000',
'--device',
'mps',
'--grid-size',
'24',
'--resolution',
'0.45',
'--sigma',
'0.5',
'--embedding-dim',
'256',
'--base-channels',
'32',
'--dropout',
'0.1',
'--lr',
'1e-3',
'--weight-decay',
'1e-6',
'--checkpoint-path',
'artifacts/voxel_autoencoder_qm9.pt',
'--artifact-path',
'artifacts/voxel_autoencoder_qm9.json']
```

```
[35]: # Uncomment to retrain the autoencoder with the stronger preset:
# run_command('train_mol2_autoencoder',
↳MOLSIM_COMMANDS['train_mol2_autoencoder'], family='molSIM')
```

```
[36]: def plot_reconstruction_metric_distribution(df: pd.DataFrame) -> go.Figure:
    fig = make_subplots(rows=1, cols=2, subplot_titles=('Voxel MSE', 'Voxel
↳Overlap @ 0.1'))
    fig.add_trace(go.Bar(x=df['sample_name'], y=df['voxel_mse'],
↳name='voxel_mse'), row=1, col=1)
    fig.add_trace(go.Bar(x=df['sample_name'], y=df['voxel_overlap_0.1'],
↳name='overlap_0.1'), row=1, col=2)
    fig.update_layout(title='Random-sample reconstruction quality',
↳showlegend=False)
    return fig

safe_show('plot_reconstruction_metric_distribution', lambda:
↳plot_reconstruction_metric_distribution(sample_eval_df))
```

1.4 Recommended Workflow

1. Run one command at a time with `run_command(...)`.
2. Confirm expected artifact files were created.

3. Re-run the plotting cells to refresh the 2D and 3D views.
4. Add a short human summary to `results/RESULTS_LOG.md`.
5. Keep large outputs under ignored paths such as `artifacts/`, `downstream/data/`, and `results/generated/`.

1.5 DegradeMaster End-To-End Walkthrough

This section is the downstream stakeholder view. It follows one PROTAC sample through: - `name.json` metadata - raw mol2 structure - voxelized representation using the selected SVE encoder preprocessing - exported SVE embedding - feature dimensionality before and after SVE merge - baseline vs with-SVE downstream metrics

```
[37]: def load_protac_name_dict(dataset_root: Path | None = None) -> dict:
    dataset_root = dataset_root or (PROJECT_ROOT / 'data' / 'PROTAC')
    path = dataset_root / 'name.json'
    if not path.exists():
        raise FileNotFoundError(f'Missing PROTAC name.json: {path}')
    return json.loads(path.read_text(encoding='utf-8'))

def choose_protac_sample(sample_id: str | None = None, seed: int = 42,
    dataset_root: Path | None = None) -> tuple[str, dict, Path]:
    dataset_root = dataset_root or (PROJECT_ROOT / 'data' / 'PROTAC')
    name_dict = load_protac_name_dict(dataset_root)
    keys = sorted(name_dict.keys(), key=lambda x: int(x) if str(x).isdigit()
    else x)
    key = sample_id if sample_id is not None else random.Random(seed).
    choice(keys)
    meta = name_dict[key]
    mol2_path = dataset_root / 'protac' / meta['protac_path']
    if not mol2_path.exists():
        raise FileNotFoundError(f'PROTAC mol2 missing: {mol2_path}')
    return key, meta, mol2_path

def load_protac_embeddings(npz_path: Path | None = None) -> dict[str, np.
    ndarray]:
    npz_path = npz_path or (PROJECT_ROOT / 'artifacts' /
    'degrademaster_protac_embeddings.npz')
    if not npz_path.exists():
        return {}
    arr = np.load(npz_path, allow_pickle=True)
    names = [str(x) for x in arr['names'].tolist()]
    embs = np.asarray(arr['embeddings'], dtype=np.float32)
    return {name: embs[i] for i, name in enumerate(names)}
```

```

def build_protac_sample_bundle(sample_id: str | None = None, seed: int = 42) -> dict:
    key, meta, mol2_path = choose_protac_sample(sample_id=sample_id, seed=seed)
    model, voxel_config, payload = load_voxel_autoencoder_from_artifact()
    coords, atomic_nums, bonds = parse_mol2_structure(mol2_path)
    voxel = voxelize_positions(coords, atomic_nums, voxel_config)
    if payload.get('_input_normalization') == 'per_sample_max':
        voxel = normalize_voxel(voxel)
    with torch.no_grad():
        recon, embedding = model(voxel.unsqueeze(0).float())
        embedding_np = embedding.squeeze(0).cpu().numpy()
        exported_embs = load_protac_embeddings()
        exported = exported_embs.get(mol2_path.stem)

        protac_feat = np.load(PROJECT_ROOT / 'data' / 'PROTAC' / 'features' /
        'protac_feature.npy', allow_pickle=True) if (PROJECT_ROOT / 'data' /
        'PROTAC' / 'features' / 'protac_feature.npy').exists() else None
        protac_feat_sve = np.load(PROJECT_ROOT / 'data' / 'PROTAC_sve' / 'features' /
        'protac_feature.npy', allow_pickle=True) if (PROJECT_ROOT / 'data' /
        'PROTAC_sve' / 'features' / 'protac_feature.npy').exists() else None
        idx = int(key) if str(key).isdigit() else None

        feature_summary = {
            'baseline_protac_dim': None if protac_feat is None else int(protac_feat.
            shape[1]),
            'sve_protac_dim': None if protac_feat_sve is None else
            int(protac_feat_sve.shape[1]),
            'sample_idx': idx,
        }

        metrics = {
            'sample_id': key,
            'sample_name': mol2_path.stem,
            'embedding_dim': int(embedding_np.shape[-1]),
            'exported_embedding_found': exported is not None,
            'voxel_max': float(voxel.max()),
            'recon_max': float(recon.max()),
            'voxel_mse': compute_voxel_mse(voxel.squeeze(0).numpy(), recon.
            squeeze(0).squeeze(0).cpu().numpy()),
            'voxel_overlap_0.1': compute_voxel_overlap(voxel.squeeze(0).numpy(),
            recon.squeeze(0).squeeze(0).cpu().numpy(), threshold=0.1),
        }

    return {
        'sample_id': key,
        'meta': meta,

```

```

    'mol2_path': mol2_path,
    'coords': coords.numpy(),
    'atomic_nums': atomic_nums.numpy(),
    'bonds': bonds,
    'voxel': voxel.squeeze(0).numpy(),
    'recon': recon.squeeze(0).squeeze(0).cpu().numpy(),
    'embedding': embedding_np,
    'exported_embedding': exported,
    'feature_summary': feature_summary,
    'metrics': metrics,
    'voxel_config': voxel_config,
}

```

```

protac_sample_bundle = build_protac_sample_bundle()
safe_show('protac_sample_metrics', lambda: pd.
↳ DataFrame([protac_sample_bundle['metrics']]))

```

	sample_id	sample_name	embedding_dim	exported_embedding_found	voxel_max \
0	1927	1315_protac	256	True	1.0

	recon_max	voxel_mse	voxel_overlap_0.1
0	0.728441	0.003638	0.543423

```

[38]: def protac_metadata_table(bundle: dict) -> pd.DataFrame:
    rows = []
    for key, value in bundle['meta'].items():
        rows.append({'field': key, 'value': str(value)})
    return pd.DataFrame(rows)

safe_show('protac_metadata_table', lambda:
↳ protac_metadata_table(protac_sample_bundle))

```

	field	value
0	pro_comp_id	1315
1	tar_path	PARP1_P09874.pdb
2	e3_ligase_path	CRBN_Q96SW2.pdb
3	e3_lig_path	CRBN_Q96SW2_2_lig_smina_out.mol2
4	war_path	PARP1_P09874_22_lig_smina_out.mol2
5	protac_path	1315_protac.mol2
6	smiles	O=C1CCC(N2C(=O)C3=CC=CC(NCCOCCOCCOCCOCCOCCOCC(...
7	label	-1

```

[39]: def plot_protac_point_cloud(bundle: dict) -> go.Figure:
    coords = bundle['coords']
    atomic_nums = bundle['atomic_nums']
    colors = [ATOM_COLORS.get(int(z), 'pink') for z in atomic_nums]

```

```

fig = go.Figure()
fig.add_trace(go.Scatter3d(x=coords[:,0], y=coords[:,1], z=coords[:,2],
↪mode='markers', marker=dict(size=5, color=colors), text=[f'Z={int(z)}' for z
↪in atomic_nums], name='atoms'))
bx, by, bz = [], [], []
for i, j in bundle['bonds']:
    bx.extend([coords[i, 0], coords[j, 0], None])
    by.extend([coords[i, 1], coords[j, 1], None])
    bz.extend([coords[i, 2], coords[j, 2], None])
if bx:
    fig.add_trace(go.Scatter3d(x=bx, y=by, z=bz, mode='lines',
↪line=dict(color='black', width=3), name='bonds'))
fig.update_layout(title=f"PROTAC point cloud:
↪{bundle['metrics']['sample_name']}", scene=dict(aspectmode='data'))
return fig

safe_show('plot_protac_point_cloud', lambda:
↪plot_protac_point_cloud(protac_sample_bundle))

```

```

[40]: def plot_protac_voxel_walkthrough(bundle: dict) -> go.Figure:
    voxel = bundle['voxel']
    recon = bundle['recon']
    threshold = suggest_reconstruction_threshold({'recon': recon, 'voxel':
↪voxel})
    orig_coords, orig_vals = voxel_to_points(voxel, threshold=threshold)
    recon_coords, recon_vals = voxel_to_points(recon, threshold=threshold)
    mid = voxel.shape[0] // 2
    fig = make_subplots(rows=2, cols=2, specs=[[{'type': 'scene'}, {'type':
↪'scene'}], [{'type': 'xy'}, {'type': 'xy'}]], subplot_titles=('Voxel cloud',
↪'Recon voxel cloud', 'Original central slice', 'Recon central slice'))
    if len(orig_coords) > 0:
        fig.add_trace(go.Scatter3d(x=orig_coords[:,0], y=orig_coords[:,1],
↪z=orig_coords[:,2], mode='markers', marker=dict(size=3, color=orig_vals,
↪colorscale='Viridis', opacity=0.7)), row=1, col=1)
    if len(recon_coords) > 0:
        fig.add_trace(go.Scatter3d(x=recon_coords[:,0], y=recon_coords[:,1],
↪z=recon_coords[:,2], mode='markers', marker=dict(size=3, color=recon_vals,
↪colorscale='Plasma', opacity=0.7)), row=1, col=2)
    fig.add_trace(go.Heatmap(z=voxel[:, :, mid], coloraxis='coloraxis'), row=2,
↪col=1)
    fig.add_trace(go.Heatmap(z=recon[:, :, mid], coloraxis='coloraxis'), row=2,
↪col=2)
    fig.update_layout(title=f"PROTAC voxel walkthrough (threshold={threshold:.
↪4f})", coloraxis=dict(colorscale='Viridis'), showlegend=False)
    return fig

```



```
safe_show('plot_protac_voxel_walkthrough', lambda:
    ↪plot_protac_voxel_walkthrough(protac_sample_bundle))
```

```
[41]: def protac_feature_dimension_table(bundle: dict) -> pd.DataFrame:
    fs = bundle['feature_summary']
    rows = [
        {'view': 'baseline', 'protac_feature_dim': fs['baseline_protac_dim']},
        {'view': 'with_sve', 'protac_feature_dim': fs['sve_protac_dim']},
    ]
    df = pd.DataFrame(rows)
    if fs['baseline_protac_dim'] is not None and fs['sve_protac_dim'] is not
    ↪None:
        df['added_sve_dim'] = [0, fs['sve_protac_dim'] -
    ↪fs['baseline_protac_dim']]
        return df

safe_show('protac_feature_dimension_table', lambda:
    ↪protac_feature_dimension_table(protac_sample_bundle))
```

	view	protac_feature_dim	added_sve_dim
0	baseline	167	0
1	with_sve	423	256

```
[42]: def plot_protac_embedding_profile(bundle: dict) -> go.Figure:
    emb = bundle['embedding']
    fig = make_subplots(rows=1, cols=2, subplot_titles=('Embedding histogram',
    ↪'Embedding line profile'))
    fig.add_trace(go.Histogram(x=emb, nbinsx=40), row=1, col=1)
    fig.add_trace(go.Scatter(x=np.arange(len(emb)), y=emb,
    ↪mode='lines+markers'), row=1, col=2)
    fig.update_layout(title=f"PROTAC embedding profile:
    ↪{bundle['metrics']['sample_name']}", showlegend=False)
    return fig

safe_show('plot_protac_embedding_profile', lambda:
    ↪plot_protac_embedding_profile(protac_sample_bundle))
```

```
[43]: def classification_metric_files() -> list[Path]:
    return sorted(PROJECT_ROOT.glob('downstream/runs_classification/*/
    ↪final_metrics.json'))

def classification_metrics_frame() -> pd.DataFrame:
```

```

rows = []
reg = load_degrademaster_results_registry()
for rec in reg.get('classification', {}).values():
    row = {'run_name': rec.get('run_name', rec.get('tag',
↳ 'classification_run'))}
    row.update(rec.get('metrics', {}))
    rows.append(row)
for path in classification_metric_files():
    payload = _load_json(path)
    if not payload:
        continue
    payload = {'run_name': path.parent.name, **payload}
    rows.append(payload)
if not rows:
    return pd.DataFrame()
return pd.DataFrame(rows).drop_duplicates(subset=['run_name'], keep='last')

safe_show('classification_metrics_frame', lambda:
↳ classification_metrics_frame())

```

	run_name	loss	accuracy	auc	f1	precision	\
2	classification_baseline	2.437170	0.747508	0.788850	0.742248	0.705426	
3	classification_with_sve	6.156187	0.574751	0.551965	0.372099	1.000000	

	recall	dataset_root	dataset_type	\
2	0.705426	/Users/isthismanas/Documents/SVEAssistedModell...	name	
3	0.007752	/Users/isthismanas/Documents/SVEAssistedModell...	name	

	feature	protac_dim
2	True	167
3	True	423

```

[44]: def classification_sve_comparison_frame() -> pd.DataFrame:
    df = classification_metrics_frame()
    if df.empty:
        return pd.DataFrame()
    metric_cols = [c for c in ['loss', 'accuracy', 'auc', 'f1', 'precision',
↳ 'recall'] if c in df.columns]
    df['comparison_key'] = df['run_name'].str.replace('_baseline', '',
↳ regex=False).str.replace('_with_sve', '', regex=False)
    df['sve_flag'] = np.where(df['run_name'].str.contains('with_sve'),
↳ 'with_sve', np.where(df['run_name'].str.contains('baseline'), 'baseline',
↳ 'other'))
    base = df[df['sve_flag'] == 'baseline'][['comparison_key', *metric_cols]].
↳ rename(columns={c: f'{c}__baseline' for c in metric_cols})

```

```

    sve = df[df['sve_flag'] == 'with_sve'][['comparison_key', *metric_cols]].
    ↪rename(columns={c: f'{c}__with_sve' for c in metric_cols})
    merged = base.merge(sve, on='comparison_key', how='inner')
    for metric in metric_cols:
        merged[f'{metric}__delta'] = merged[f'{metric}__with_sve'] -
    ↪merged[f'{metric}__baseline']
    return merged

safe_show('classification_sve_comparison_frame', lambda:
    ↪classification_sve_comparison_frame())

```

```

    comparison_key  loss__baseline  accuracy__baseline  auc__baseline \
0  classification          2.43717          0.747508          0.78885

    f1__baseline  precision__baseline  recall__baseline  loss__with_sve \
0      0.742248          0.705426          0.705426          6.156187

    accuracy__with_sve  auc__with_sve  f1__with_sve  precision__with_sve \
0          0.574751          0.551965          0.372099          1.0

    recall__with_sve  loss__delta  accuracy__delta  auc__delta  f1__delta \
0          0.007752          3.719016          -0.172757          -0.236885          -0.370149

    precision__delta  recall__delta
0          0.294574          -0.697674

```

```

[45]: def plot_classification_sve_deltas(metric: str = 'accuracy') -> go.Figure:
    df = classification_sve_comparison_frame()
    if df.empty:
        raise ValueError('No paired classification baseline/with-SVE runs found
    ↪yet.')
    col = f'{metric}__delta'
    fig = go.Figure(data=[go.Bar(x=df['comparison_key'], y=df[col], text=np.
    ↪round(df[col], 6), textposition='outside')])
    fig.update_layout(title=f'Classification with-SVE minus baseline:
    ↪{metric}', xaxis_title='run family', yaxis_title='delta')
    return fig

# safe_show('plot_classification_sve_deltas', lambda:
    ↪plot_classification_sve_deltas('accuracy'))

```

```

[46]: def degrademaster_curve_files() -> list[Path]:
    return sorted((PROJECT_ROOT / 'results' / 'generated' /
    ↪'degrademaster_curves').glob('*.json'))

```

```

def degrademaster_curve_frame(task: str | None = None) -> pd.DataFrame:
    rows = []
    for path in degrademaster_curve_files():
        payload = _load_json(path)
        if not payload:
            continue
        if task is not None and payload.get('task') != task:
            continue
        run_name = str(payload.get('run_name', path.stem))
        for row in payload.get('history', []):
            rows.append({'run_name': run_name, 'task': payload.get('task'),
↳**row})
    return pd.DataFrame(rows)

safe_show('degrademaster_classification_curve_frame', lambda:
↳degrademaster_curve_frame(task='classification').head())
safe_show('degrademaster_regression_curve_frame', lambda:
↳degrademaster_curve_frame(task='regression').head())

```

Empty DataFrame

Columns: []

Index: []

Empty DataFrame

Columns: []

Index: []

```

[47]: def plot_classification_learning_curves(metric: str = 'val_auc') -> go.Figure:
    df = degrademaster_curve_frame(task='classification')
    if df.empty or metric not in df.columns:
        raise ValueError(f'No classification curve data available for {metric}')
    fig = go.Figure()
    for run_name, sub in df.groupby('run_name'):
        fig.add_trace(go.Scatter(x=sub['epoch'], y=sub[metric],
↳mode='lines+markers', name=run_name))
    fig.update_layout(title=f'Classification learning curves: {metric}',
↳xaxis_title='epoch', yaxis_title=metric)
    return fig

# safe_show('plot_classification_learning_curves', lambda:
↳plot_classification_learning_curves('val_auc'))

```

```

[48]: def plot_regression_learning_curves(metric: str = 'mean_rmse') -> go.Figure:
    df = degrademaster_curve_frame(task='regression')
    if df.empty or metric not in df.columns:

```

```

        raise ValueError(f'No regression curve data available for {metric}')
    fig = go.Figure()
    for run_name, sub in df.groupby('run_name'):
        fig.add_trace(go.Scatter(x=sub['epoch'], y=sub[metric],
mode='lines+markers', name=run_name))
    fig.update_layout(title=f'Regression learning curves: {metric}',
axis_title='epoch', yaxis_title=metric)
    return fig

# safe_show('plot_regression_learning_curves', lambda:
plot_regression_learning_curves('mean_rmse'))

```

1.6 Final Takeaway

This section summarizes the strongest currently supported downstream conclusion from the completed runs. It intentionally avoids claiming formal statistical significance without repeated seeds, but it does identify whether SVE improved any completed architectures and whether the current best regression model is SVE-based.

```

[49]: def final_takeaway_table() -> pd.DataFrame:
    reg = downstream_metrics_frame().copy()
    if reg.empty or 'mean_rmse' not in reg.columns:
        return pd.DataFrame()
    reg = reg.sort_values('mean_rmse').reset_index(drop=True)
    reg['is_sve'] = reg['run_name'].str.contains('with_sve')
    reg['rank'] = np.arange(1, len(reg) + 1)
    keep = [c for c in ['rank', 'run_name', 'is_sve', 'mean_rmse',
'rmse_dc50_nm', 'rmse_dmax_pct', 'r2_dc50_nm', 'r2_dmax_pct'] if c in reg.
columns]
    return reg[keep]

def final_takeaway_text() -> str:
    reg = downstream_metrics_frame().copy()
    if reg.empty or 'mean_rmse' not in reg.columns:
        return 'No completed regression runs are available yet.'
    reg = reg.sort_values('mean_rmse').reset_index(drop=True)
    best = reg.iloc[0]
    paired = downstream_sve_comparison_frame()
    improved = []
    if not paired.empty and 'mean_rmse_delta' in paired.columns:
        improved = paired.loc[paired['mean_rmse_delta'] < 0, 'comparison_key'].
tolist()
    sve_phrase = 'SVE-based' if 'with_sve' in str(best['run_name']) else
'non-SVE baseline'

```

```

    improvement_phrase = ', '.join(improved) if improved else 'no completed_
paired regression variants yet'
    return (
        f"Best completed regression model so far: {best['run_name']}_
({sve_phrase}) with mean RMSE={best['mean_rmse']:.3f}. "
        f"Completed paired comparison suggests SVE improved these regression_
variants: {improvement_phrase}. "
        f"This supports the claim that SVE can improve downstream regression_
quality in an architecture-dependent way, and the current best regression_
model is SVE-based."
    )

safe_show('final_takeaway_table', lambda: final_takeaway_table())
print(final_takeaway_text())

```

	rank	run_name	is_sve	mean_rmse \
0	1	regression_cross_attention_with_sve	True	222.999043
1	2	regression_base_baseline	False	234.242956
2	3	regression_cross_attention_baseline	False	239.079641
3	4	regression_two_head_with_sve	True	242.368694
4	5	regression_base_with_sve	True	243.492614
5	6	regression_two_head_baseline	False	248.461138
6	7	regression_tabular_baseline	False	274.388972
7	8	regression_tabular_with_sve	True	274.388972
8	9	regression_pdc50_bounded_with_sve	True	336.705016
9	10	regression_pdc50_bounded_baseline	False	504.309527
10	11	classification_baseline	False	NaN
11	12	classification_with_sve	True	NaN

	rmse_dc50_nm	rmse_dmax_pct	r2_dc50_nm	r2_dmax_pct
0	431.129632	14.868455	0.251040	0.125557
1	452.548902	15.937011	0.174772	-0.004647
2	463.262836	14.896447	0.135235	0.122261
3	469.393139	15.344249	0.112197	0.068697
4	470.358185	16.627044	0.108543	-0.093528
5	481.148937	15.773338	0.067171	0.015882
6	534.484408	14.293537	-0.151100	0.191874
7	534.484408	14.293537	-0.151100	0.191874
8	651.966594	21.443437	-0.712749	-0.818815
9	987.175616	21.443437	-2.926737	-0.818815
10	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN

Best completed regression model so far: regression_cross_attention_with_sve (SVE-based) with mean RMSE=222.999. Completed paired comparison suggests SVE improved these regression variants: regression_cross_attention, regression_pdc50_bounded, regression_two_head. This supports the claim that SVE

can improve downstream regression quality in an architecture-dependent way, and the current best regression model is SVE-based.